THINGS TO KNOW :

1. **Lab report must contain following sections: (order must be maintained)**
   a) **Title /Question**
   b) **Theory: The brief overview of the concept /techniques/algorithm used in the program**
   c) **Code: The complete code**
   d) **Output: Screenshot of the output**
2. **Output screen should be captured (use snipping tool), printed and attached in the report. Other contents must be handwritten.**
3. **Every Source code must include the printing statements to print following information after your main output:**

   Lab No.: Name: Roll No./Section :

4. **Contents should be written on single side of A4 sized paper.**
5. **The works must be submitted within specified deadline.**
6. **Cover page and contents page should be attached in the report appropriately.**

==========

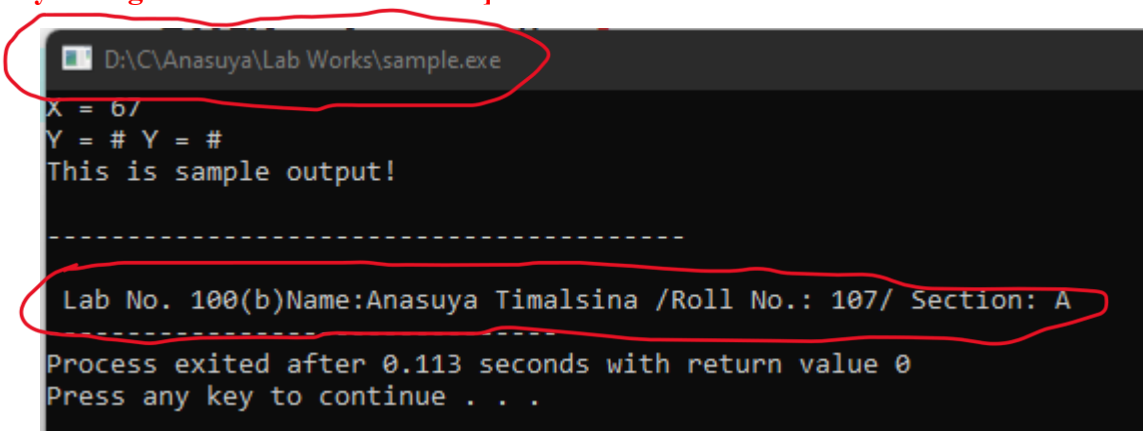
Contents Page Format (can be printed)

List of Lab Works

Lab No.	Title /Question	Submission Date	Signature	Remarks
1(a)	To print bio data	2078/08/22		
1(b)	To take two numbers and display their sum	2078/08/22		

Sample output:

[show the top of window of output screen showing directory structure of your name and your signature line at the bottom]



Lab Works (Part -2)

10. **Symbol Table Construction and Scope Management**
    Implement a symbol table in C++ using a stack of hash maps to handle nested scopes for a simple block-structured language (e.g., supporting variables, functions, and blocks like { ... }). The symbol table should store identifier names, types (int, float, char), addresses, and scope levels. Write a semantic analyzer that traverses a given abstract syntax tree (AST) from a provided parser output, inserts entries, checks for redeclarations in the same scope, and resolves references with scope rules. Test with 3 sample programs including nested blocks and report any semantic errors like undeclared variables or scope violations.
    [Idea: Use std::unordered_map for efficient lookups and a std::vector of maps for scoping; include functions for enter_scope(), leave_scope(), insert(), and lookup().]
11. **Type Checking for Expressions**
    In C, develop a semantic analysis module that performs type checking on arithmetic expressions and assignments. Support types like int, float, bool, and arrays. Given a parsed AST, recursively evaluate types, enforce compatibility (e.g., no int + string), and perform implicit type conversions where applicable (e.g., int to float). Handle array indexing to check bounds semantically if sizes are known. Test with 4 valid and 4 invalid code snippets, outputting type errors with line numbers. Idea: Define an enum for types and a recursive function that returns the type of each node; use a symbol table from Question 10 to fetch variable types, and simulate errors with custom structs for error reporting.
12. **Semantic Rules for Control Structures**
    Using C++, implement semantic checks for if-else, while, and for loops in a mini-language. Ensure conditions are boolean-typed, check for break/continue only in loops, and verify variable initialization in for loops. Integrate with a symbol table to track initialized variables and prevent use-before-definition errors. Provide a driver program that reads a source file, parses it (assume a simple lexer/parser provided), and runs semantic analysis. Include 5 test cases with mixed valid/invalid semantics.

[Idea: Build on an AST visitor pattern; use flags in symbol entries for initialization status, and propagate errors up the tree for comprehensive reporting.]

13. Intermediate Code Generation

Write a C++ program to generate three-address code (TAC) for arithmetic and relational expressions, including operators like +, -, *, /, ==, >, and unary minus. Input an infix expression string, parse it to postfix using a stack, then convert to TAC using temporary variables. Output the code in quadruple format (op, arg1, arg2, result). Test with 6 expressions of increasing complexity, including parentheses.

[Idea: Use std::stack for parsing; assign temps like t1, t2 incrementally; handle operator precedence carefully to avoid errors in code generation.]

14. TAC Generation for Control Flow Statements

In C, implement intermediate code generation for if-else and while statements. Given a parsed AST, generate TAC with labels for jumps (e.g., if cond goto L1; ... L1:). Support nested structures and backpatching for forward references in conditions. Include boolean short-circuit evaluation for && and ||. Test with 4 sample control flow programs and display the generated TAC list.

[Idea: Use a label counter for unique labels; maintain lists of true/false exits for backpatching; represent TAC as a struct array with fields for opcode, operands, and result.]

15. Intermediate Code for Functions and Parameters

Using C++, generate TAC for function definitions, calls, and parameter passing (by value). Handle return statements and ensure parameter count/type matching semantically (integrate basic type checking). Input a simple program AST, output TAC with activation records simulated via labels. Test with 3 functions including recursion, and note any generation challenges in a report.

[Idea: Use special opcodes like PARAM, CALL, RETURN; track function symbols in a table; simulate stack for parameters to mimic runtime behavior in code.]

16. Code Generation

Implement a code generator in C++ that translates TAC quadruples into x86 assembly code for arithmetic operations. Assume a simple register model with 4 registers (e.g., eax, ebx, ecx, edx). Allocate registers naively (e.g., fixed assignment) and handle spills to memory if needed. Input TAC from previous assignments, output assembly with MOV, ADD, SUB, etc. Test with 5 TAC examples and simulate execution if possible.

[Idea: Map TAC ops to assembly instructions; use a register descriptor table to track free/busy registers; include prologue/epilogue for main function.]

17. Constant Folding and Propagation

Implement an optimizer in C for TAC that performs constant folding (e.g., replace 2+3 with 5) and propagation (e.g., if x=5, replace later x uses with 5). Traverse the TAC list multiple passes until no changes. Handle only arithmetic constants. Test with 5 unoptimized TACs, show before/after, and measure instruction reduction.

[Idea: Use a value table to track known constants; for each quadruple, check if args are constants and compute; propagate by substituting in subsequent quads.]

18. Dead Code Elimination

In C++, write an optimizer to remove dead code from TAC, identifying unreachable code after jumps and unused assignments. Use a mark-and-sweep approach: mark live quads backward from end, sweep unmarked. Also eliminate redundant assignments. Test with 4 TAC examples including loops, report optimized code.
[Idea: Build a control flow graph (CFG) with basic blocks; perform liveness analysis per block; represent CFG as adjacency list of quad indices].

19. Peephole Optimization

Using C, apply peephole optimization on assembly code from previous generators. Look for patterns like redundant MOVs (e.g., MOV eax, ebx; MOV ebx, eax → remove), or ADD 0 → NOP. Scan a window of 2-3 instructions, replace matches. Test with 6 assembly snippets, show optimizations applied. Idea: Read assembly as string array; define pattern rules as structs with match/replace functions; apply iteratively for multiple passes.